

Nikhil Savant

312-989-8206 | nsavant033@gmail.com | [linkedin.com/in/nikhil-savant](https://www.linkedin.com/in/nikhil-savant) | U.S. Citizen

EDUCATION

University of Illinois Urbana-Champaign

Bachelor of Science in Computer Science and Economics

Urbana-Champaign, IL

Expected Graduation: 2028

Coursework: Database Systems, Algorithms and Models of Computation, Computer Systems, Data Structures

TECHNICAL SKILLS AND CLUBS

Languages: Java, Python, C++, C, Go, TypeScript, JavaScript, R, HTML/CSS, Swift, SQL

Frameworks: LitJS, Node.js, Spring Boot, NextJS, React, Angular, React Native

Developer Tools: Linux, CoreML, AWS, Jupyter, Git, MongoDB, Docker, Figma, Nginx

Libraries: Pandas, NumPy, OpenCV, Tkinter, SK-Learn, TensorFlow

Clubs: UIUC ACM Math and Algorithms Member, UIUC Engineering Council Activities Coordinator

WORK

Cisco, San Jose, CA

Software Engineering Intern

May - Aug 2026

- Designed MCP tools for inventory lookup, fault analysis, object search, and account/org-aware summaries, targeting a **40%** reduction in time spent manually navigating operational data
- Integrated an API entry point for MCP traffic, enabling secure routing, CORS handling, trace propagation, and gateway-level controls for **10K+** projected daily AI-assisted requests.
- Built registration and catalog workflows for new MCP servers, supporting **50+** registered tools with JSON schema validation, service labels, health checks, and dynamic cache refresh across gateway replicas.
- Designed a centralized MCP integration layer to aggregate tool discovery and execution across **8+** downstream MCP servers, exposing a unified interface for AI applications.

Algo Analytics

Full Stack Engineer Intern

Jun - Oct 2025

- Developed a TypeScript based backend integration layer to connect the React Native client with financial API's, aggregating real-time stock data and reducing client-side parsing times by **30%**
- Implemented RESTful API controllers to serialize and sanitize high-throughput financial data streams, successfully processing **100,000+** data points daily while achieving a **300ms** API response time
- Developed scalable backend services that utilized JWT authentication and Axios interceptors for secure session management, optimizing JSON payload sizes by **20%** and ensuring zero-downtime token refresh cycles

StellarPay

Software Engineering Intern

Jun - Aug 2025

- Built a GPT-powered financial assistant in TypeScript/Node.js, leveraging a Retrieval-Augmented Generation (RAG) system with engineered prompts to deliver precise, context-aware insights
- Refactored front-end codebase and components improving component reusability and load times leading to a reduction of code duplication by **25%** and a reduction in load times by **20%**
- Enhanced retrieval precision by **40%** using AWS Textract for document ingestion and OpenAI Embeddings with Pinecone for high-performance semantic search
- Implemented end-to-end security protocols across AWS services, including IAM role-based access control, encrypted data storage with AWS KMS (AES-256), and API Gateway authorization layers using JWTs to safeguard sensitive user data

PROJECTS

Custom Git Server

- Created a custom Git server in Go capable of remote cloning, fetching, and pushing via the Smart HTTP and SSH protocols, allowing direct handling of TCP/IP socket connections
- Developed a secure authentication pipeline integrating EdDSA SSH key validation and JWTs, ensuring strict process isolation and role-based access control
- Designed an efficient streaming parser to read raw Git object packs and parse DAGs, capping buffer memory at under **30MB** for large repository transfers

Container Runtime

- Developed a custom, Go-based container runtime to securely isolate and execute application workloads
- Utilized Linux namespaces (PID, NET, MOUNT) and cgroups via Go system calls to isolate application processes, accurately capping CPU and memory usage to handle **50+** concurrent workloads without performance degradations
- Implemented a layered file system utilizing OverlayFS to optimize container initialization, achieving startup times under **100ms** and reducing storage redundancies by **80%** by sharing base image layers across instances